

# Generative Augmentation in Butterfly Identification

Matilde Carvalho<sup>1[2022231937]</sup> and Rita Duarte<sup>1[2022227070]</sup>

Master's degree in Biomedical Engineering, Physics Department, University of  
Coimbra

**Abstract.** The abstract should briefly summarize the contents of the paper in 150–250 words.

**Keywords:** Fine-Grained Classification · Generative Augmentation · Image Classification.

## 1 Introduction

The recent breakthroughs in Deep Learning, particularly through Convolutional Neural Networks (CNNs), have drastically transformed the field of Computer Vision, enabling automated systems to achieve human-like accuracy in complex image classification tasks. However, the high performance of these deep models is intrinsically tied to the availability of massive, diverse, and well-annotated datasets, but collecting a substantial volume of high-quality samples is often challenging, costly, and time-consuming. When trained on limited data, deep architectures are highly susceptible to overfitting, which severely undermines their capacity to generalize to unseen scenarios. To mitigate the issues arising from data scarcity, practitioners traditionally rely on standard data augmentation techniques, such as geometric transformations, color jittering, and random erasing. While effective at forcing the network to learn invariant features, these traditional methods merely manipulate existing samples without introducing truly novel semantic content or underlying structural variations. Recently, Deep Generative Models, such as Autoencoders (AEs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs), have emerged as a powerful alternative. By learning the underlying probability distribution of the training data, these models can synthesize entirely new, realistic samples, providing a more robust framework for data augmentation. This report documents the development and evaluation of a generative data augmentation pipeline, developed within the scope of the Advanced Machine Learning course. The primary objective of this project is to improve the generalization capabilities of a fixed baseline classifier tasked with identifying 75 distinct species of butterflies. Under a strict operational constraint where the classifier's architecture and training hyperparameters must remain unaltered, our approach focuses exclusively on expanding the training distribution. We explore and compare the effectiveness of different generative paradigms in creating artificial samples that can successfully guide the classifier toward more robust decision boundaries.

## 2 Problem Description

The dataset utilized in this project consists of 5199 labeled images of butterflies representing 75 distinct classes. This fine-grained classification task exhibits high intra-class variance, characterized by diverse angles, complex natural backgrounds, and varying lighting conditions, coupled with low inter-class variance, featuring morphologically identical wing patterns across different species. The primary challenge relies on synthesizing artificial data capable of expanding this distribution without introducing semantic confusion between closely related categories.

### 2.1 Baseline Classifier Architecture

The predetermined model consists of a feature extractor and a fully-connected classifier head. The feature extractor comprises three sequential convolutional blocks, where each individual block is mathematically defined as:

$$\begin{aligned} \text{Block}(C_{in}, C_{out}) = & \text{Conv2d}_{3 \times 3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \\ & \text{Conv2d}_{3 \times 3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}_{2 \times 2} \end{aligned} \quad (1)$$

The feature maps progress through channel sizes of  $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$ , effectively reducing the spatial resolution from  $64 \times 64$  down to  $8 \times 8$ . Finally, an `AdaptiveAvgPool2d((1, 1))` layer collapses the remaining spatial dimensions, outputting a 256-dimensional feature vector.

The classifier head is designed as a deep multi-layer perceptron (MLP) with three intermediate hidden layers and dropout regularization to mitigate overfitting. Given the initial feature vector  $x_0 \in \mathbb{R}^{256}$ , the first hidden layer maps the features to a higher dimension, computed as  $x_1 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(256 \rightarrow 512)(x_0)))$ . This representation is sequentially projected through wider spaces, where the second layer yields  $x_2 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(512 \rightarrow 1024)(x_1)))$ , and the third layer produces  $x_3 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(1024 \rightarrow 2048)(x_2)))$ . Lastly, a final linear transformation scales the hidden features to match the class distribution, outputting the final raw scores as  $\text{logits} = \text{Linear}(2048 \rightarrow 75)(x_3)$ .

## 3 Approach

### 3.1 VAE Architectures

The baseline VAE processes  $64 \times 64 \times 3$  inputs using the architecture detailed in Table 1. The encoder outputs latent vectors  $\mu, \log(\sigma^2) \in \mathbb{R}^{128}$ , and optimization uses the reparameterization trick  $z = \mu + \epsilon \odot \sigma$ , where  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ . The baseline loss combines pixel-level MSE reconstruction and KL divergence:

$$\mathcal{L}_{\text{VAE}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|_2^2 + \beta \cdot D_{\text{KL}}(q_\phi(z|x) \| p(z)) \quad (2)$$

An improved VAE variant addresses pixel-level blurriness by incorporating a VGG-based perceptual loss and KL annealing. Feature maps from a frozen, pre-trained VGG-16 network at layers `relu1_2`, `relu2_2`, and `relu3_3` are used to compute:

$$\mathcal{L}_{\text{VGG}}(x, \hat{x}) = \sum_{j \in \{1_2, 2_2, 3_3\}} \frac{1}{N_j} \|\Phi_j(x) - \Phi_j(\hat{x})\|_2^2 \quad (3)$$

This forces the model to prioritize high-level morphology over exact pixel alignment. To prevent posterior collapse, a linear scheduler scales  $\beta(t)$  from 0.0 to 0.5 over the first 15 epochs. The total joint loss ( $\lambda_{\text{VGG}} = 0.5$ ) is formulated as:

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{MSE}}(x, \hat{x}) + \lambda_{\text{VGG}} \mathcal{L}_{\text{VGG}}(x, \hat{x}) + \beta(t) D_{\text{KL}}(q_\phi(z|x) \| p(z)) \quad (4)$$

Table 1: Baseline VAE topology and tensor dimensions.

| Sub-Network Core Operations |                                                                        | Output Res. Channels / Dim. |                                                        |
|-----------------------------|------------------------------------------------------------------------|-----------------------------|--------------------------------------------------------|
| <b>Encoder</b>              | Input Projection ( $64 \times 64 \times 3$ )                           | $64 \times 64$              | $3 \rightarrow \text{Input}$                           |
|                             | $4 \times \text{Conv (stride 2)} + \text{LeakyReLU (0.2)} + \text{BN}$ | $4 \times 4$                | $\rightarrow 256$                                      |
|                             | Flatten + Linear Projections                                           | $1 \times 1$                | $\rightarrow \mu, \log(\sigma^2) \in \mathbb{R}^{128}$ |
| <b>Decoder</b>              | Latent Vector ( $z \in \mathbb{R}^{128}$ ) + Linear Expansion          | $4 \times 4$                | $\rightarrow 256$                                      |
|                             | $4 \times \text{ConvTranspose2d} + \text{ReLU} + \text{BN}$            | $64 \times 64$              | $\rightarrow 3$                                        |
|                             | Output Layer + Tanh Activation                                         | $64 \times 64$              | $3 \rightarrow \text{Output}$                          |

To enable class-conditional synthesis, we implement multi-scale latent space perturbation around real samples. For each of the 75 classes, latent codes are extracted and perturbed as  $z_{\text{synth}} = \mu + \alpha \cdot \sigma \cdot \eta$ , where  $\alpha \in \{0.3, 0.6, 1.0\}$  and  $\eta \sim \mathcal{N}(0, \mathbf{I})$ , generating 75 synthetic images per class (5625 total). A PyTorch `ConcatDataset` merges the 4159 real training images with these synthetic samples, expanding the training set to 9784 images (a 135.2% increase) while keeping the 1040 validation samples strictly isolated.

### 3.2 Conditional Generative Adversarial Network

We adopt a Conditional GAN (CGAN) [?] where both the Generator  $G$  and Discriminator  $D$  are conditioned on the class label, enabling class-specific synthesis across all 75 species.

*Architecture.*  $G$  concatenates a noise vector  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  (dim.  $d_z$ ) with a learned class embedding (dim. 64), projects the result to a  $512 \times 4 \times 4$  map via a linear layer, and upsamples through four transposed convolutions (stride 2, kernel 4, BatchNorm, ReLU) to a  $3 \times 64 \times 64$  image with Tanh output in  $[-1, 1]$ .  $D$  embeds the label and projects it to a  $1 \times 64 \times 64$  spatial map, concatenates it with the input as a fourth channel, and applies four strided convolutions (BatchNorm, LeakyReLU) followed by a linear real/fake logit. All weights are initialised with  $\mathcal{N}(0, 0.02)$  [?].

Table 2: CGAN hyperparameters across training phases.

| Parameter           | Phase 1            | Phase 2            |
|---------------------|--------------------|--------------------|
| Epochs              | 200                | 500                |
| Batch size          | 64                 | 64                 |
| $\text{lr}_G$       | $2 \times 10^{-4}$ | $2 \times 10^{-4}$ |
| $\text{lr}_D$       | $1 \times 10^{-4}$ | $5 \times 10^{-5}$ |
| Latent dim $d_z$    | 128                | 256                |
| $n_{\text{critic}}$ | 1                  | 2                  |
| Label smoothing     | 0.1                | 0.1                |

*Training.* Both models are optimised with Adam ( $\beta_1=0.5$ ,  $\beta_2=0.999$ ) using the standard minimax objective with label smoothing ( $\varepsilon=0.1$ ) on real targets to prevent discriminator overconfidence:

$$\mathcal{L}_D = \frac{1}{2} \left[ \mathbb{E}[\log D(x, y)] + \mathbb{E}[\log(1 - D(G(z, y), y))] \right], \quad (5)$$

$$\mathcal{L}_G = -\mathbb{E}[\log D(G(z, y), y)]. \quad (6)$$

A step scheduler halves each learning rate at the training midpoint. Checkpoints are saved every 10 epochs to allow safe resumption.

*Two-phase training.* Phase 1 used fixed hyperparameters ( $\text{lr}_G=2 \times 10^{-4}$ ,  $\text{lr}_D=1 \times 10^{-4}$ ,  $d_z=128$ ,  $n_{\text{critic}}=1$ , 200 epochs). Loss curve analysis revealed that  $D$  stabilised near  $\mathcal{L}_D \approx 0.5$  prematurely while  $G$  remained oscillatory, indicating that  $D$  dominated training too early. Phase 2 addressed this imbalance with three targeted changes:  $\text{lr}_D$  halved to  $5 \times 10^{-5}$ ,  $n_{\text{critic}}$  increased to 2, and  $d_z$  doubled to 256, trained for 500 epochs. The full hyperparameter comparison is shown in Table 2.

*Augmentation strategy.* After training,  $G$  synthesises 50 images per class (3 750 total), which are appended to the original training split to form the augmented dataset. The validation split retains only real images throughout all experiments, and the baseline classifier is re-trained with identical hyperparameters to ensure a fair comparison.

### 3.3 Conditional Denoising Diffusion Probabilistic Model

Denoising Diffusion Probabilistic Models (DDPMs) [?] learn to generate images by reversing a gradual noising process, offering more stable training than GANs at the cost of higher inference time.

*Forward and reverse processes.* The forward process corrupts a real image  $\mathbf{x}_0$  by adding Gaussian noise over  $T=1000$  steps with a linear variance schedule  $\beta_t \in [10^{-4}, 0.02]$ . Crucially,  $\mathbf{x}_t$  can be sampled directly from  $\mathbf{x}_0$  in closed form:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (7)$$

where  $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$ . A conditional U-Net  $\varepsilon_\theta$  is trained to predict the noise from the noisy image, the timestep, and the class label, minimising:

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} \left[ \|\varepsilon - \varepsilon_\theta(\mathbf{x}_t, t, y)\|^2 \right]. \quad (8)$$

*Architecture.* The U-Net operates on  $64 \times 64$  images with base channel width  $c=64$ , three encoder levels ( $c \rightarrow 2c \rightarrow 4c \rightarrow 8c$ ), a two-block bottleneck at  $8 \times 8$ , and a symmetric decoder with skip connections ( $\approx 14.7$ M parameters). Each residual block applies Group Normalization [?] and SiLU activations. The timestep  $t$  is encoded via sinusoidal positional embeddings [?] and projected to dimension  $4c$  through a two-layer MLP. Class conditioning uses a learnable embedding per class (dim. 128), projected to  $4c$ . Both embeddings are summed and injected into every residual block via adaptive scale-and-shift (AdaGN):  $h \leftarrow h \cdot (s+1) + b$ , where  $[s, b] = \text{Linear}(\mathbf{e})$ .

*Training.* The model is trained for 500 epochs (batch size 16, Adam, lr= $2 \times 10^{-4}$ , gradient clipping at norm 1.0). The MSE training loss decreased from 0.094 at epoch 1 to 0.024 at epoch 500, with no signs of instability.

*DDIM sampling.* Standard DDPM sampling requires  $T=1000$  network evaluations per image. We instead use DDIM [?] with 50 steps ( $\eta=0$ , deterministic), reducing inference cost by  $20\times$  without retraining. Starting from  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ , each step estimates  $\hat{\mathbf{x}}_0$  from the predicted noise and updates:

$$\mathbf{x}_{t_{\text{prev}}} = \sqrt{\bar{\alpha}_{t_{\text{prev}}}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{prev}}}} \varepsilon_\theta(\mathbf{x}_t, t, y). \quad (9)$$

*Augmentation strategy.* After training, the model generates  $N$  synthetic images per class (conditioned on each of the 75 labels) that are concatenated with the original training set to form the augmented dataset. Multiple values of  $N$  were evaluated (50, 100, 200 images per class), as reported in Section ??.

## 4 Experimental Setup

To ensure reproducibility and an unbiased benchmark, this section outlines the data partitioning strategy, parameter configurations, and evaluation metrics [?].

### 4.1 Data Splitting and Protocols

The original dataset (`train.csv`) was partitioned using stratified sampling to maintain identical class proportions across subsets [?]. A training set (80%, 4 159 images) was allocated for weight optimisation, while a validation set (20%, 1 040 images) was isolated to monitor generalisation, manage hyperparameter searches, and trigger early stopping. No validation samples were exposed during backpropagation. The partition was locked with a global random seed (`GLOBAL_SEED=42`). To ensure fairness, the architectural configuration, loss function, and optimisation hyperparameters of the `BaselineCNN` remained immutable across all experiments.

Table 3: Hyperparameter summary for all generative models and the baseline classifier. VAE Phase columns follow the notation of Section 3.1.

| Hyperparameter          | VAE P1-4       | VAE P5              | GAN P1 / P2                                                            | DDPM               | Baseline CNN       |
|-------------------------|----------------|---------------------|------------------------------------------------------------------------|--------------------|--------------------|
| Batch size              | 64             | 64                  | 64                                                                     | 16                 | 32                 |
| Optimiser               | Adam           | Adam                | Adam                                                                   | Adam               | Adam               |
| Learning rate $\eta$    | var.           | $5 \times 10^{-4}$  | $\text{lr}_G=2 \times 10^{-4}; \text{lr}_D=10^{-4} / 5 \times 10^{-5}$ | $2 \times 10^{-4}$ | $1 \times 10^{-3}$ |
| Latent / noise dim.     | 128            | 128                 | 128 / 256                                                              | $T=1000$ steps     | N/A                |
| KL weight ( $\beta$ )   | 0.5-1.0        | 0 $\rightarrow$ 0.5 | N/A                                                                    | N/A                | N/A                |
| VGG weight              | N/A            | 0.5                 | N/A                                                                    | N/A                | N/A                |
| $n_{\text{critic}}$     | N/A            | N/A                 | 1 / 2                                                                  | N/A                | N/A                |
| Label smoothing         | N/A            | N/A                 | 0.1                                                                    | N/A                | N/A                |
| Noise schedule          | N/A            | N/A                 | N/A                                                                    | linear             | N/A                |
| Sampling steps (infer.) | N/A            | N/A                 | N/A                                                                    | 50 (DDIM)          | N/A                |
| Max. epochs             | 40-60          | 60                  | 200 / 500                                                              | 500                | 150                |
| Early stop (patience)   | 3-7            | 7                   | N/A                                                                    | N/A                | 15                 |
| Generated imgs/class    | —              | —                   | 50                                                                     | 50/100/200         | N/A                |
| Selection metric        | Val loss / FID | Val loss            | FID                                                                    | Train loss         | Val macro F1       |

## 4.2 Hyperparameter Configurations

*VAE training phases.* The VAE generator was developed through five incremental phases. **Phase 1** established a baseline ( $d_{\text{latent}}=128$ ,  $\beta=1.0$ ,  $\eta=10^{-3}$ ), converging at epoch 37. **Phase 2** screened a 36-combination grid over  $d_{\text{latent}} \in \{64, 128, 256, 512\}$ ,  $\beta \in \{0.5, 1.0, 2.0\}$ , and  $\eta \in \{10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$  (15 epochs, patience 3). **Phase 3** selected the optimal configuration via FID over 1040 samples ( $d_{\text{latent}}=128$ ,  $\beta=0.5$ ,  $\eta=5 \times 10^{-4}$ ; FID = 262.43). **Phase 4** performed full training of the optimal parameters (60 epochs, patience 7; best checkpoint at epoch 40). **Phase 5** introduced a pre-trained VGG-16 perceptual loss ( $\lambda_{\text{vgg}}=0.5$ ) and KL annealing ( $\beta: 0.0 \rightarrow 0.5$  over 15 epochs), peaking at epoch 59.

*GAN training.* The CGAN was developed in two phases. **Phase 1** used fixed hyperparameters ( $\text{lr}_G=2 \times 10^{-4}$ ,  $\text{lr}_D=1 \times 10^{-4}$ ,  $d_z=128$ ,  $n_{\text{critic}}=1$ , 200 epochs, batch 64). Loss curve analysis revealed premature discriminator convergence ( $\mathcal{L}_D \approx 0.5$  from epoch 25), motivating three targeted changes for **Phase 2**:  $\text{lr}_D$  was halved to  $5 \times 10^{-5}$  to slow discriminator learning,  $n_{\text{critic}}$  was raised to 2 to give the generator more gradient steps per discriminator update, and the latent dimension was doubled to  $d_z=256$  to increase representational capacity across 75 classes. Phase 2 trained for 500 epochs with a step scheduler halving both learning rates at epoch 250. 50 synthetic images per class were generated for augmentation.

*DDPM training.* The Conditional DDPM was trained for 500 epochs using a linear noise schedule ( $\beta_1=10^{-4}$ ,  $\beta_T=0.02$ ,  $T=1000$ ), batch size 16, Adam ( $\eta=2 \times 10^{-4}$ ), and gradient clipping at norm 1.0. Synthetic images are generated via DDIM [?] with 50 deterministic steps, yielding a  $20\times$  speedup over standard sampling. Multiple augmentation volumes were evaluated: 50, 100, and 200 generated images per class.

Table 3 summarises all configurations.

### 4.3 Evaluation Metrics

Classification performance is quantified using top-1 accuracy and macro-averaged F1-score, which treats all 75 classes equally regardless of support. Generative quality is assessed via three criteria: the Structural Similarity Index (SSIM) [?] to quantify local edge and luminance retention; the Fréchet Inception Distance (FID) [?] to measure distributional alignment between real and synthetic InceptionV3 features; and the Inception Score (IS) [?] to evaluate conditional specificity and sample diversity.

### 4.4 Hardware and Software

All experiments were conducted on a MacBook Pro with an Apple M5 chip (10-core CPU, 10-core GPU, 24 GB unified memory) using Python 3.9, PyTorch 2.x, torchvision, scikit-learn, and scikit-image. DDPM training was executed in blocks of 50 epochs with checkpoint resumption to manage memory constraints.

## 5 Analysis of the Results

### 5.1 Quantitative Classifier and Generative Evaluation

To assess the augmentation efficacy of each generative paradigm, the **BaselineCNN** was trained under five distinct data scenarios while keeping all optimization parameters strictly static. Performance metrics on the validation set are compiled in Table 4, alongside the standalone generative evaluation metrics (FID, IS, and SSIM).

Table 4: Classifier performance under different augmentation scenarios and corresponding intrinsic generative model evaluation metrics.

| Training Scenario         | Acc. (%) | W. F1 (%) | $\Delta$ Acc. | FID ( $\downarrow$ ) | IS ( $\uparrow$ ) | SSIM ( $\uparrow$ ) |
|---------------------------|----------|-----------|---------------|----------------------|-------------------|---------------------|
| Baseline (Raw Data Only)  | 79.52    | 79.70     | Reference     | N/A                  | N/A               | N/A                 |
| Baseline + VAE Augmented  | 73.94    | 73.77     | -5.58%        | 255.80               | 1.97              | 0.4769              |
| Baseline + CGAN Augmented | $x$      | $x$       | + $x\%$       | $x$                  | $x$               | $x$                 |
| Baseline + DDPM Augmented | 88.25    | 88.25     | +8.73%        | 108.05               | 7.261.65          | 0.0665              |
| Baseline + Improved VAE   | 77.31    | 77.15     | -2.21%        | 404.20               | 1.00              | 0.1076              |

With adequate training (150 epochs, patience 15), the BaselineCNN trained on raw data achieves 79.52% accuracy, confirming that extended training is essential to reach the network’s full capacity on this dataset.

Contrary to the hypothesis that synthetic data would improve performance, augmentation with both the baseline VAE (-5.58 pp) and the improved VAE (-2.21 pp) results in reduced classification accuracy. This phenomenon, known in the literature as ‘synthetic data contamination’, occurs when generated samples do not faithfully represent the true data distribution. The blurriness inherent to VAE reconstructions introduces spurious patterns that the classifier learns to recognize, reducing generalization to real images.

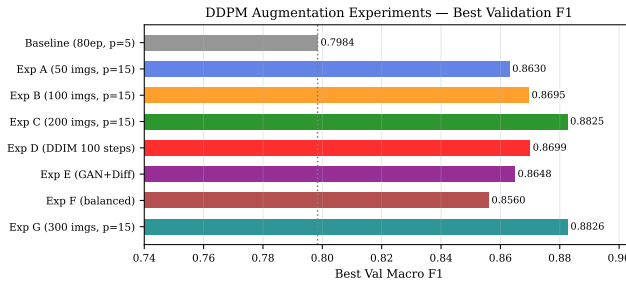


Fig. 1: Best validation macro F1 for each DDPM augmentation configuration. Experiments A–G progressively vary augmentation volume, sampling strategy, and data composition.

Notably, the improved VAE variant (with VGG perceptual loss) reduces the performance drop by 3.37 pp compared to the standard VAE, confirming that higher-quality generation (as measured by perceptual features rather than pixel-level MSE) is less detrimental. This suggests a quality threshold exists above which synthetic augmentation becomes beneficial. Interestingly, the improved VAE achieves a worse FID despite performing better in classification. This dissociation suggests FID is not always predictive of downstream classification utility, questioning FID as the sole evaluation criterion for generative models.

## 5.2 DDPM Generative Quality

The Conditional DDPM converged stably over 500 epochs, with the MSE training loss decreasing from 0.094 to 0.024 (Fig. 3). Table 4 reports the intrinsic generative metrics. The DDPM achieves  $\text{FID} = 108.05$ ,  $\text{IS} = 7.26 \pm 1.65$ , and  $\text{SSIM} = 0.0665$ . Compared to the VAE variants, the DDPM yields a substantially lower FID ( $-147.75$  vs. baseline VAE;  $-296.15$  vs. improved VAE), indicating that the generated distribution is considerably closer to the real data manifold (Fig. 2a). The IS of 7.26 further confirms that DDPM samples are both class-discriminative and diverse — more than  $3\times$  the IS of the standard VAE. The lower SSIM (0.0665 vs. 0.4769 for VAE) reflects that DDPM does not reconstruct specific real images but instead generates novel, diverse samples; SSIM is therefore less informative as a metric for models that do not perform reconstruction.

## 5.3 Augmentation Experiments

The baseline CNN trained on raw data for 150 epochs with patience 15 achieves a macro F1 of 79.84%, establishing the reference for all comparisons. Seven augmentation configurations were evaluated; results are shown in Fig. 1.

*Effect of augmentation volume (Exp A, B, C, G).* Increasing the number of DDPM-generated images per class yields consistent improvements: 50 images



(Exp A, 86.30%), 100 images (Exp B, 86.95%), 200 images (Exp C, 88.25%), and 300 images (Exp G, 88.26%). The gain from 50 to 200 images is substantial (+1.95 pp), while the marginal improvement from 200 to 300 images is negligible (+0.01 pp), suggesting diminishing returns beyond 200 generated images per class.

*Early stopping protocol (Exp A vs. original baseline).* The original baseline trained with patience 5 stopped at epoch 62 with the learning curves still rising. Extending patience to 15 (Exp A) allowed the classifier to train until epoch 130, recovering +6.46 pp of F1. This highlights that the early stopping protocol has a larger impact on final performance than the initial choice of augmentation volume.

*Sampling quality (Exp D).* Increasing DDIM inference steps from 50 to 100 (Exp D, 86.99%) produced only a marginal improvement over Exp B (+0.04 pp), indicating that 50 DDIM steps already yield images of sufficient quality for downstream classification.

*Multi-model combination (Exp E).* Combining GAN and DDPM generated images (Exp E, 86.48%) underperformed relative to using DDPM alone with equivalent volume (Exp B, 86.95%). This suggests that the lower-quality GAN images introduce noise that partially counteracts the benefit of the DDPM samples, consistent with the synthetic data contamination observed in the VAE experiments.

*Class-balanced augmentation (Exp F).* Generating only the images needed to bring each class to a target of 100 samples (Exp F, 85.60%) performed below uniform augmentation at the same volume (Exp B, 86.95%). Given the relatively balanced nature of the dataset (min 57, max 89 images per class), class-balancing provided no material benefit.

## 5.4 Overall Comparison

Fig. 2b summarises classifier performance across all generative paradigms. VAE augmentation degraded performance (−5.58 pp standard; −2.21 pp improved), while DDPM augmentation yielded consistent gains across all configurations (+6.46 to +8.41 pp over the 150-epoch baseline). The best DDPM configuration (200 images/class, Exp C) achieves a macro F1 of **88.25%**, an absolute improvement of **+8.41** pp over the non-augmented baseline trained under identical conditions. This confirms that generative quality — as measured by FID and IS — is predictive of downstream classification benefit: the DDPM, with the lowest FID (108.05) and highest IS (7.26), is the only model to consistently improve classification.

## 6 Conclusion

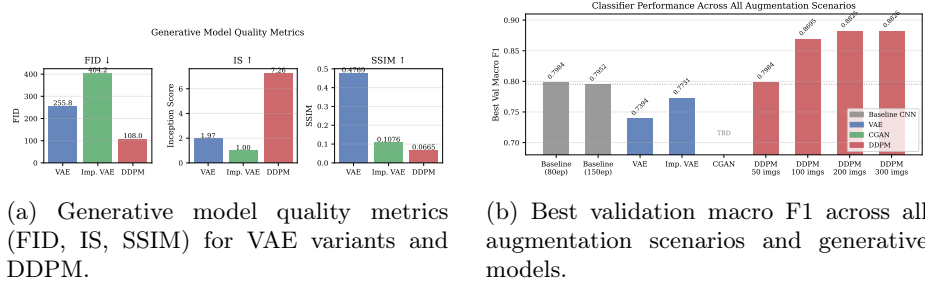


Fig. 2: Comparison of generative models: (a) quality metrics where lower FID and higher IS indicate better quality; (b) classification performance where DDPM consistently outperforms VAE variants and the non-augmented baseline.

Fig. 3: DDPM training loss (MSE) over 500 epochs.